INFORMATION TECHNOLOGY UNIVERSITY (ITU), LAHORE

Department of Data Science

ASSIGNMENT 2 DDPM on AFHQ Cats

Course: Generative AI & Large Language Models (LLMs)
Semester: Fall 2026

Submitted By : Fazeelat Maheen
Program: MS Data Science

Roll Number: MSDS-25040

Submitted To: Dr. Mohsin Ali

Submission Date : November 8, 2026

1. Introduction

A Diffusion Model learns to generate new images by gradually adding noise to images and then learning how to remove it step by step.

In this assignment (Q6), I implemented a **Denoising Diffusion Probabilistic Model (DDPM)** in `diffusion.py`. It includes a cosine noise schedule, forward diffusion, training loss, and reverse sampling. I also implemented a **U-Net architecture** in `unet.py` to predict and remove the added noise.

The model was trained on **32 × 32 AFHQ Cat images** using a single Kaggle T4 GPU.

This report covers the implementation and experimental setup, training loss (Q6.1), FID score (Q6.2), generated images (Q6.3), and forward and reverse diffusion visualizations (Q6.4 and Q6.5). It also includes limitations, answers to Q1–Q5, and an AI tools disclosure.

2. Method

2.1 Forward Diffusion Process

The forward process gradually adds Gaussian noise to a clean image ((x_0)) over **T = 50 steps**. This makes the image noisier at each step. Instead of adding noise step by step, we can directly calculate the noisy image ((x_t)) from the original image in one step using the following equation:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim N(0, I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t$ is the cumulative product of the α_s:

$$\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$$

The coefficient $\sqrt{\bar{\alpha}_t}$ says how much of the original image is left, and $\sqrt{1 - \bar{\alpha}_t}$ how much noise has been mixed in. At t = 0 the image is untouched; as $\bar{\alpha}_t \to 0$ the image turns into pure noise.

2.2 Cosine Noise Schedule

Instead of using a fixed linear noise schedule, I used a **cosine schedule** with (s = 0.008). It controls how noise is added at each step and helps maintain a smoother noise distribution throughout the diffusion process. The individual beta values are calculated from this schedule using the following equations:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \qquad f(t) = cos^2\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right), \qquad \beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}$$

2.3 Training Process

During training, the model takes a clean image ((x_0)), selects a random timestep ((t)), and adds Gaussian noise to create a noisy image ((x_t)). The U-Net then predicts the noise added to the image.

The model is trained by minimizing the **L1 loss**, which measures the difference between the actual noise and the predicted noise.

$$L = E_{x_0, t, \epsilon} \left\| \epsilon - \epsilon_\theta\left(x_t, t\right) \right\|_1$$

Predicting the noise gives the network a target with the same scale at every timestep, and since x_0 can be recovered from the predicted noise (next section), nothing is lost.

2.4 Sampling (Reverse Diffusion Process)

The reverse process starts with random Gaussian noise and gradually converts it into an image. At each step, the U-Net predicts the noise, which is then removed to make the image clearer. This process continues from the final timestep back to the first step until a new image is generated.

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

This estimate is clamped to [−1, 1] so pixel values stay valid and errors do not grow. The posterior mean of x_{t-1} is then a weighted mix of the current image and the clamped estimate:

$$\mu = \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}\,(1-\alpha_t)}{1-\bar{\alpha}_t}\,\hat{x}_0$$

Finally fresh noise is added, except at the last step:

$$x_{t-1} = \mu + \sigma_t z, \qquad z \sim N(0, I) \;\; (z = 0 \; at \; the \; last \; step)$$

2.5 U-Net Architecture

The model uses a U-Net with a base width of 16 and channel multipliers of [1, 2, 4, 8], giving 16, 32, 64, and 128 channels.

The downsampling path reduces image size while increasing the number of channels. The middle block processes the features, and the upsampling path restores the image to 32 × 32.

Skip connections transfer information between matching downsampling and upsampling layers, helping preserve image details. The timestep is also converted into a sinusoidal embedding and passed through an MLP, allowing the network to understand the noise level at each step.

3. Experimental setup

Setting	Value
Dataset	AFHQ cats (train split)
Image size	32 × 32
Diffusion steps T	50
Noise schedule	Cosine, s = 0.008
Loss	L1 on predicted noise
Batch size	32 (2 gradient-accumulation steps)
Learning rate	1e-3
U-Net	dim 16, dim multipliers [1, 2, 4, 8]
Hardware	Kaggle, NVIDIA T4 GPU
FID	clean-fid, 512 generated samples vs 500 real cat validation images
Training length	1,000 steps (run reported here)
FID evaluation	Every 100 training steps

3. Experimental Setup and Results

The results presented in this report are mainly from the 1,000-step training run, with FID calculated after every 100 steps. The FID score was measured using Clean-FID, with 512 generated images compared against 500 real AFHQ cat validation images.

Other experiments included a 1,000-step run without FID evaluation and a 10,000-step training run. Their configurations and results are listed below.

- 1,000-step run (without FID): [Add save_and_sample_every value]
- 10,000-step run: [Add save_and_sample_every value]
- WandB run: [Add run name or link]
- Saved model (model.pt): [Insert Google Drive link]

4. Q6.1 Training loss



During the 1,000 training steps, the L1 loss decreased from **0.851 to 0.168**, showing that the U-Net gradually improved its ability to predict the added noise. This represents a reduction of approximately 80%.

The loss decreased more quickly during the early stages of training and became more stable over time. Small fluctuations were expected because the model used random timesteps and noise during training.

5. Q6.2 FID

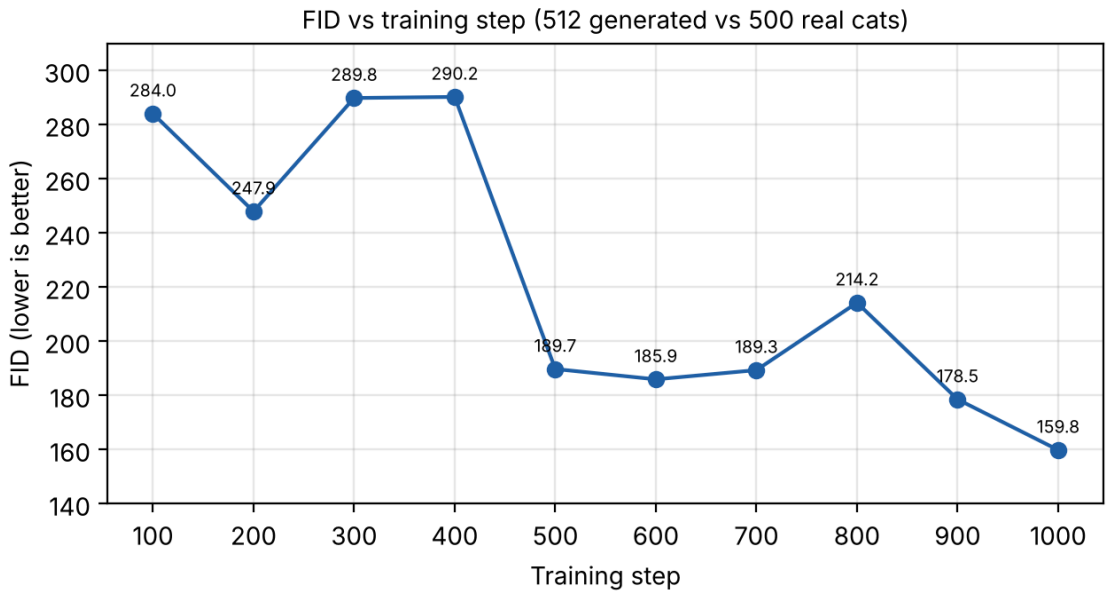


Figure 1: FID every 100 training steps (re-plotted from the logged values).

FID was calculated every 100 training steps using 512 generated images and 500 real AFHQ cat validation images.

Overall Trend: The FID score decreased from 284.0 at step 100 to 159.8 at step 1,000. This shows an overall improvement in the generated image quality. The best score was achieved at the final checkpoint.

Fluctuations: The FID score did not decrease continuously. It increased at some checkpoints, especially between steps 200 and 400 and again between steps 600 and 800. These changes are expected because the model is still learning and the generated samples can vary.

Limitations of FID: Since only 512 generated images and 500 real images were used, the FID estimates may be noisy and less reliable. Therefore, small differences between checkpoints should be interpreted carefully.

Overall Performance: The final FID score of 159.8 is still relatively high. This may be due to the small U-Net architecture, limited training steps, 32 × 32 image resolution, and 50-step sampling process. The model has learned to generate images, but their quality is still limited.

6. Q6.3 Generated images

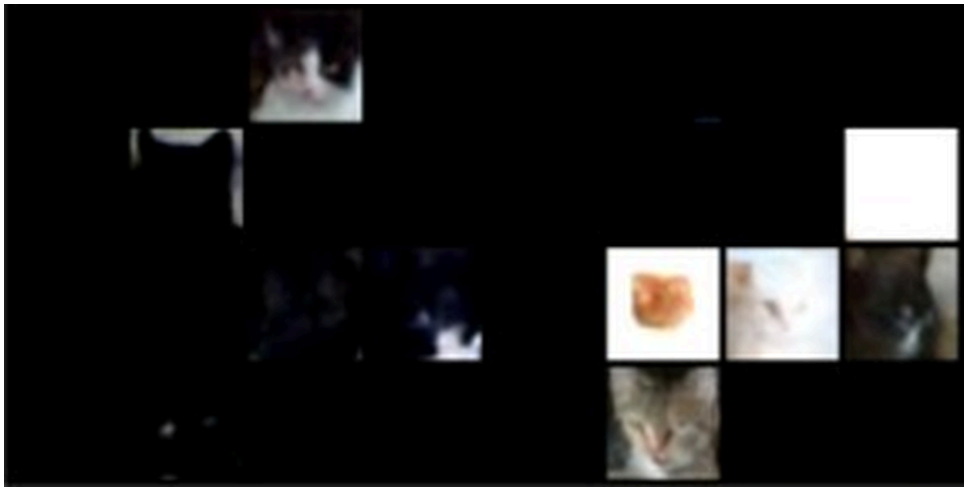


Figure 2: samples saved by the notebook (results/q6_3/sample_9.png), as displayed in Kaggle.

The generated samples show recognizable cat faces in different colors, including orange, grey, white, and dark tabby. Some images have visible ears, eyes, and noses, although the details are still blurry.

Some tiles in Figure 2 appear black, while one is completely white. These seem to be display issues rather than actual model outputs, as the samples in Figure 4 are displayed correctly.

The final samples show that the model has learned basic cat-like features, but fine details such as fur texture and whiskers are still missing. This is consistent with the relatively high FID score.

Figure 2: Generated cat samples from the model.

Note: Make sure Figure 2 contains the correct final 1,000-step output (`q6_3_generated_cats.png`). If `sample_9.png` belongs to another run, replace it with the correct image before submission.

7. Q6.4 Forward diffusion



Figure 3: forward diffusion of 8 training cats at 0%, 25%, 50%, 75% and 99% of T = 50 (t = 0, 12, 25, 37, 49).

Approximate values from the cosine formula with T = 50 are given in brackets as $\sqrt{\bar{\alpha}_t}$ (image part) and $\sqrt{1 - \bar{\alpha}_t}$ (noise part).

The forward process gradually adds Gaussian noise to the original cat images over 50 steps.

- **0% (t = 0):** Original cat images are clearly visible.
- **25% (t = 12):** Noise appears, but cat faces and colors remain recognizable.
- **50% (t = 25):** Noise becomes stronger, leaving only a few basic image features.
- **75% (t = 37):** Almost all image details disappear.
- **99% (t = 49):** The images become almost completely random Gaussian noise.

The cosine schedule ensures gradual noise addition while eventually converting the original images into almost pure noise. This provides the starting point for the reverse diffusion process.

8. Q6.5 Backward diffusion

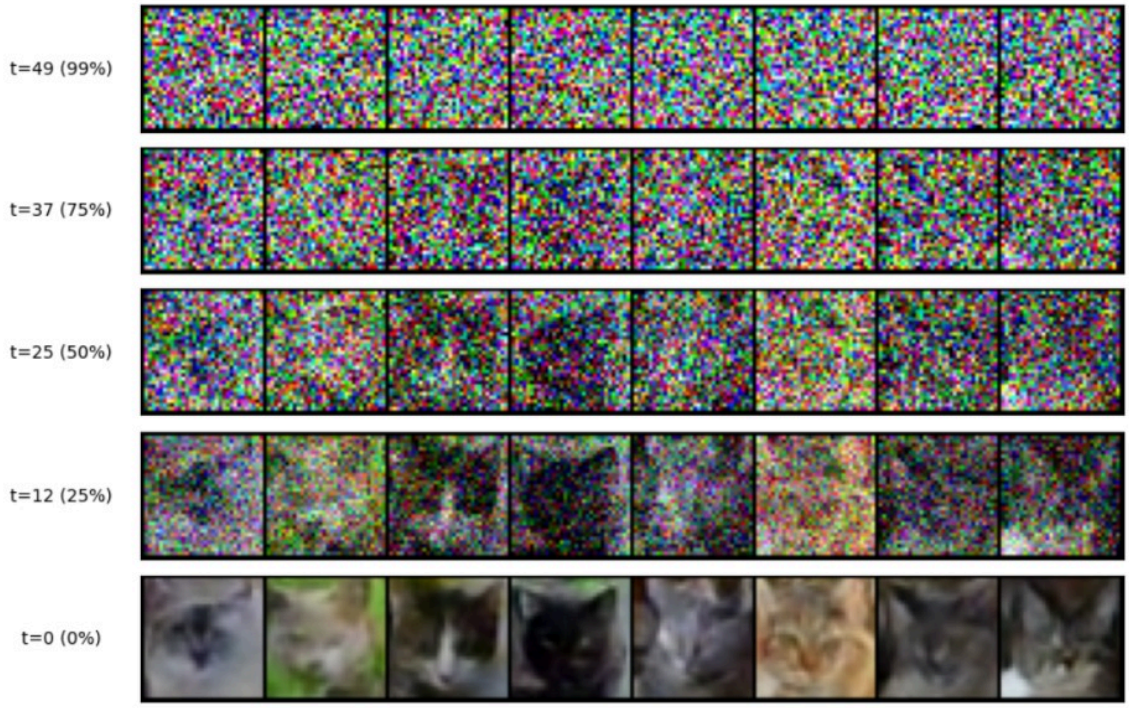


Figure 4: backward diffusion of 8 samples from t = 49 (99%) down to t = 0 (0%).

Reverse Diffusion Process – Image Generation

The reverse diffusion process starts with random noise and gradually generates cat images by removing noise at each timestep.

Observations at Different Diffusion Stages

99% and 75% (t = 49 and 37):
The images mainly contain random noise, with no clear structure or recognizable features.

50% (t = 25):
Some light and dark patterns begin to appear in the images.

25% (t = 12):
Faint cat-like features, including faces, ears, and light-colored patches, become visible.

0% (t = 0):
Eight slightly blurry but recognizable cat faces are generated in different colors, including grey, orange, black, and white.

Conclusion

The generated images are different from the original training images because the reverse diffusion process starts from fresh random noise. Therefore, the outputs are new generated samples rather than reconstructions of the original cat images

9. Observations and Limitations

Limitations of the Model

The model faced several limitations that influenced its overall performance and the quality of the generated images.

1. Small Network Architecture

The U-Net uses a base width of 16 channels, which restricts its ability to capture complex features and produce fine details such as fur texture and whiskers.

2. Limited Training Steps

The model was trained for only 1,000 steps. The observed loss and FID values indicate that additional training may help improve image quality and model performance.

3. Low Image Resolution

Images were generated at a resolution of 32 × 32 pixels, which limits the level of visual detail. Furthermore, the diffusion process used only 50 timesteps.

4. Limited FID Evaluation Samples

The FID score was calculated using 512 generated images and 500 real images. This relatively small sample size may lead to less consistent evaluation results.

5. Visualization Problems

Some outputs from the starter-code visualization appeared as black frames or dark tiles, reducing the clarity of certain figures, particularly Figure 2. In contrast, Figure 4 displays the generated samples more clearly and provides a better representation of the model's output.

10. Conclusion

Conclusion

I developed and trained a small DDPM using a cosine noise schedule on **32 × 32 AFHQ** cat images. After completing **1,000 training steps**, the training loss dropped from **0.851 to 0.168,** while the FID score decreased from **284.0 to 159.8**, indicating an improvement in the generated image quality.

The model successfully learned to produce recognizable cat faces, although the generated images still lack sharpness and fine details. These results suggest that the model has learned basic image patterns but has not yet reached its full potential. Further improvements could be achieved by using a **larger U-Net architecture**, increasing the training duration, and evaluating the model with a larger number of real and generated images.

11.QUESTIONS 1–5

QUESTION 1: CONVOLUTIONAL NEURAL NETWORKS (8 Marks)

1.1.a. Padding

To maintain the original height and width of the output, zero padding is required.

Left padding:

$p_{\square} = \lfloor K_w / 2 \rfloor$

Right padding:

$p_r = \lfloor (K_w - 1) / 2 \rfloor$

Top padding:

$p_{\square} = \lfloor K_h / 2 \rfloor$

Bottom padding:

$p_\beta = \lfloor (K_h - 1) / 2 \rfloor$

1.1.b. Convolution using the padded input

After adding zero padding, the new input is:

$x' \in \mathbb{R}^{(C \times (N_h + p_{□} + p_{\beta}) \times (N_w + p_{□} + p_r))}$

The convolution becomes:

$y'_{1,□,v} = \theta_0 + \sum_{c=1 \text{ to } C} \sum_{i=1 \text{ to } K_{□}} \sum_{j=1 \text{ to } K_v} \theta_{c,i,□} \, x'_{c,□',□'}$

where:

$m' = h - \lfloor K_{□} / 2 \rfloor + (i - 1) + p_{□}$

$n' = w - \lfloor K_v / 2 \rfloor + (j - 1) + p_{□}$

Here, the added pixels have value zero. Therefore, all indices refer to valid positions in x′, and the output shape remains:

$y' \in \mathbb{R}^{(1 \times N_{□} \times N_v)}$

1.1.c.i. Role of skip connections in U-Net

Skip connections transfer important features from the encoder to the decoder. They preserve spatial details that may be lost during downsampling and help produce accurate segmentation boundaries.

1.1.c.ii. Spatial resolution

- **Encoder:** Spatial resolution decreases through downsampling, while feature channels generally increase.
- **Decoder:** Spatial resolution increases through upsampling, while feature channels generally decrease.

QUESTION 2: ENCODER-ONLY TRANSFORMERS (4 Marks)

2.1. Directed graphical model

A decoder-only Transformer generates words one at a time. Each word depends on the previous words.

The probability distribution is:

$p(w_1, w_2, ..., w_N) = p(w_1) \times \prod_{n=2}^{N} p(w_n \mid w_1, ..., w_{n-1})$

For example, the graphical model for five words is: $w_1 \rightarrow w_2 \rightarrow w_3 \rightarrow w_4 \rightarrow w_5$

2.2. Encoder-only versus decoder-only Transformers

Encoder-only Transformers:

Encoder-only Transformers use bidirectional attention. Each word can obtain information from the entire input sequence.

For part-of-speech tagging:

$p(y_1, ..., y_N \mid w_1, ..., w_N) = \prod_{n=1}^{N} p(y_n \mid w_1, ..., w_N)$

They are suitable for:

- Part-of-speech tagging
- Text classification
- Named entity recognition
- Sentiment analysis

Decoder-only Transformers:

Decoder-only Transformers use causal attention. Each word is predicted using only the preceding words.

$p(w_1, ..., w_N) = \prod_{n=1}^{N} p(w_n \mid w_1, ..., w_{n-1})$

They are suitable for:

- Text generation
- Language modeling
- Chatbots
- Autoregressive text generation

Main difference: Encoder-only models are mainly used to understand input text, whereas decoder-only models are mainly used to generate text.

QUESTION 3: GENERATIVE ADVERSARIAL NETWORKS (5 Marks)

3.1.a. Reconstructed image

The reconstructed image combines the original unmasked pixels with the generated masked pixels.

$x' = (1 - m) \odot x + m \odot g\theta(x, m)$

Here, $\odot$ represents element-wise multiplication.

3.1.b. Mean squared error loss

Only the masked pixels should contribute to the reconstruction loss.

$\ell_{e}(\theta) = [\sum_{i} m_{i} (g\theta(x,m)_{i} - x_{i})^2] / [\sum_{i} m_{i}]$

This calculates the average squared error over the masked pixels.

3.1.c. GAN-style objective

The generator tries to produce realistic images that can fool the discriminator.

$\ell gan(\theta) = -\log d\phi(x')$

where:

$x' = (1 - m) \odot x + m \odot g\theta(x, m)$

The discriminator distinguishes real images from generated images, while the generator tries to make its reconstructed images appear realistic.

3.1.d. Disadvantage of using only MSE loss

MSE loss can produce blurry or unrealistic images because it focuses on minimizing pixel differences. The adversarial loss encourages more realistic details and textures.

QUESTION 4: VARIATIONAL AUTOENCODERS (6 Marks)

4.1. Gradient flow in a VAE

(a) Black-box sampling

When sampling is treated as a black-box operation, gradients cannot pass through the sampling node to the encoder.

Therefore, only the decoder parameters receive gradients:

w_3, b_3

(b) Reparameterization trick

Using the reparameterization trick:

$z = \mu\phi(x) + \sigma\phi(x) \odot \varepsilon$

where:

$\varepsilon \sim N(0, I)$

The sampling operation becomes differentiable with respect to the encoder parameters.

Therefore, all six parameters receive gradients:

w_1, b_1, w_2, b_2, w_3, b_3

4.2. Numerical loss for a batch of videos

The reconstruction loss for one video is:

$Lrec,i = (1 / 3FHW) \sum_{c,f,h,w} (x_{i,c,f,h,w} - \tilde{x}_{i,c,f,h,w})^2$

The KL divergence between the encoder distribution and the standard normal prior is:

$KL_i = ½ [\mu\phi(x_i)^2 + \sigma^2\phi(x_i) - \log \sigma^2\phi(x_i) - 1]$

Therefore, the total batch loss is:

$L = (1/N) \sum(i=1 \text{ to } N) [Lrec,i + \beta \times KL_i]$

This loss combines reconstruction error with KL divergence regularization.

4.3. Effect of increasing beta

Correct option: Option 1

The distribution $q\phi(z|x)$ approaches N(0, I), but reconstructions become worse.

As β increases, the KL divergence penalty becomes stronger. The latent distribution moves closer to the standard normal prior, which can reduce reconstruction accuracy.

QUESTION 5: UNDERSTANDING DIFFUSION MODELS (14 Marks)

5.1. Derivation of the ELBO decomposition

Starting with the ELBO:

$\log p(x_0) \geq E_q [\log (p\theta(x_0) / q(x_1 \mid x_0))]$

Expanding the reverse process:

$p\theta(x_0) = p(xT) \prod(t=1 \text{ to } T) p\theta(x_{t-1} \mid x_t)$

Expanding the forward process:

$q(x_1 \mid x_0) = \prod(t=1 \text{ to } T) q(x_t \mid x_{t-1})$

Substituting these expressions and separating the reconstruction term gives:

$\log p(x_0) \geq E_q(x_1|x_0) [\log p\theta(x_0 \mid x_1)] - \sum(t=1 \text{ to } T-1) L_t + C$

where:

$L_t = E_q(x_{t-1},x_t,x_{t+1}|x_0) [\log (q(x_t \mid x_{t-1}) / p\theta(x_t \mid x_{t+1}))]$

Here, C represents terms that do not depend on θ. This gives the required ELBO decomposition.

5.2. Effect of the term L□

The term L□ measures the difference between the forward transition distribution and the learned reverse transition distribution.

Since L□ is subtracted in the ELBO, maximizing the ELBO requires minimizing L□.

It is minimized when the two distributions match, giving:

L□ = 0

This encourages the reverse process to approximate the correct denoising transitions.

5.3. Sampling at timestep t = T

The forward diffusion equation is:

$q(x□ \mid x□_{-1}) = N(x□; \sqrt{α□}\, x□_{-1}, (1 - α□)I)$

Using the reparameterization trick:

$x□ = \sqrt{α□}\, x□_{-1} + \sqrt{(1 - α□)}\, ε□$

where $ε□ \sim N(0, I)$.

To obtain x_T, start from x_0 and repeatedly apply this equation for T steps.

Time complexity:

$O(T)$

5.4. Derivation using the reparameterization trick

Starting with:

$x□ = \sqrt{α□}\, x□_{-1} + \sqrt{(1 - α□)}\, ε□$

For the first timestep:

$x_1 = \sqrt{α_1}\, x_0 + \sqrt{(1 - α_1)}\, ε_1$

For the second timestep:

$x_2 = \sqrt{\alpha_2}\, x_1 + \sqrt{(1 - \alpha_2)}\, \varepsilon_2$

Substituting x_1:

$x_2 = \sqrt{(\alpha_1\alpha_2)}\, x_0 + \sqrt{(\alpha_2(1 - \alpha_1))}\, \varepsilon_1 + \sqrt{(1 - \alpha_2)}\, \varepsilon_2$

Since independent Gaussian variables form a Gaussian linear combination, this can be written as:

$x_2 = \sqrt{(\bar{\alpha}_2)}\, x_0 + \sqrt{(1 - \bar{\alpha}_2)}\, \varepsilon$

where:

$\bar{\alpha}_2 = \alpha_1\alpha_2$

Repeating this process for t timesteps gives:

Final expression:

$x_? = \sqrt{(\bar{\alpha}_?)}\, x_0 + \sqrt{(1 - \bar{\alpha}_?)}\, \varepsilon$

where:

$\bar{\alpha}_? = \prod_{i=1 \text{ to } t} \alpha_i$

and $\varepsilon \sim N(0, I)$.

Therefore:

$q(x_? \mid x_0) = N(x_?; \sqrt{\bar{\alpha}_?}\, x_0, (1 - \bar{\alpha}_?)I)$

5.5. Direct sampling at timestep t = τ

Using the derived equation, we can obtain the noisy sample directly:

$x_\tau = \sqrt{\bar{\alpha}_\tau}\, x_0 + \sqrt{(1 - \bar{\alpha}_\tau)}\, \varepsilon$

where:

$\bar{\alpha}_\tau = \prod_{i=1 \text{ to } \tau} \alpha_i$

and $\varepsilon \sim N(0, I)$.

Steps:

1. Retrieve the precomputed value of $\bar{\alpha}_t$.
2. Generate Gaussian noise ε.
3. Apply the direct sampling equation.
4. Return x_t.

Time complexity:

O(1), when $\bar{\alpha}_t$ is already precomputed.

If the cumulative product must be calculated from scratch, the complexity is O(t).

DDPM DIFFUSION ASSIGNMENT 2

FINAL SUBMISSION LINKS

1. GITHUB REPOSITORY – SOURCE CODE

https://github.com/fazeelatmaheen919435-oss/DDPM-Diffusion-Assignment-2

2. GOOGLE DRIVE – ASSIGNMENT FILES

https://drive.google.com/drive/folders/1E3qy-kxFR5-ZjX2L1qEKMmIL9I9n7uHU?usp=sharing

3. CHATGPT – AI ASSISTANCE CONVERSATION

https://chatgpt.com/share/6af03f99-e7cc-83ee-9542-abf332abc556

-